

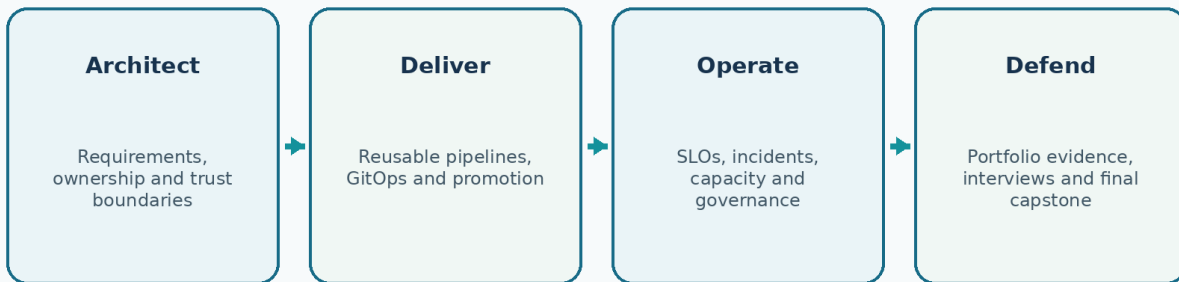
DEVOPS MASTER COURSE

Volume 10F

Enterprise Delivery Projects, Troubleshooting & Interview Mastery

Architecture • Platform Projects • Brownfield Migration • Incident Response •
Reliability • Certifications • Final Delivery Factory

Chapters 56-70 | 2026 Edition

VOLUME 10F DELIVERY SYSTEM

Design → Build → Verify → Promote → Operate → Recover → Improve

Production objective: demonstrate that you can design, build, operate, troubleshoot, secure, improve, and explain an enterprise software-delivery platform—not merely write isolated pipeline files.

About This Part

Part F is the graduation volume for CI/CD, Jenkins, GitHub Actions, GitLab CI/CD, Kubernetes delivery, GitOps, and DevSecOps. It converts the capabilities from Parts A-E into portfolio-grade platform projects, brownfield migration, disaster recovery, complex incident response, performance and capacity engineering, product management, certification strategy, technical interviews, and a final enterprise software-delivery factory.

Graduation principle: a production claim is credible only when another engineer can reproduce the evidence, understand the trade-off, observe the operating behavior, and execute the rollback or recovery procedure.

Volume 10 Roadmap

Part	Coverage	Status
10A	CI/CD foundations and pipeline engineering	Complete
10B	Jenkins administration and enterprise engineering	Complete
10C	GitHub Actions and GitLab CI/CD enterprise engineering	Complete
10D	Kubernetes delivery, GitOps and progressive delivery	Complete
10E	DevSecOps and software-supply-chain security	Complete
10F	Enterprise projects, troubleshooting and final capstone	This book

Enterprise Lab Safety and Evidence Standard

- Run destructive, load, failover, compromised-runner, credential, and deployment exercises only in approved nonproduction environments with stop conditions.
- Use synthetic data, sanitized logs, short-lived identities, private test registries, and bounded network access.
- Create budgets, quotas, expiry automation, cleanup owners, backup, rollback, RTO/RPO, and evidence custody before high-scale or recovery labs.
- Never expose employer code, production identifiers, customer data, tokens, private keys, unredacted configurations, or internal incident details in portfolio artifacts.
- A successful tool execution is not service validation; independently test security, reachability, data integrity, user experience, observability, and recovery.

How to Use This Book

1. Treat every chapter as a scenario: define outcomes, constraints, risks, owners, acceptance criteria, and evidence before implementation.
2. Build the smallest safe vertical slice, test normal and failure paths, then expand scope.
3. Create an incident or change record for each lab and preserve source, workflow, identity, artifact, approval, deployment, verification, and cleanup evidence.
4. Use dashboards, queues, cost, lead time, change failure, recovery, and adoption data to improve the platform.
5. Turn completed work into sanitized case studies with diagrams, ADRs, code, runbooks, results, trade-offs, and lessons.

Part Learning Outcomes

- Design the enterprise delivery platform as a product with explicit ownership, trust boundaries, SLOs, cost, and recovery.
- Select and measure projects from operational evidence, user research, risk, and business value.
- Standardize delivery contracts across Jenkins, GitHub Actions, GitLab CI/CD, Kubernetes, and GitOps.
- Migrate brownfield pipelines, operate release trains, and recover delivery control planes during failure or compromise.
- Troubleshoot complex controller, runner, identity, artifact, deployment, and supply-chain incidents using evidence.
- Demonstrate senior capability through certifications, system design, interviews, portfolio projects, and the final delivery-factory capstone.

Table of Contents

Chapter 56	Enterprise CI/CD Architecture and Operating Model
Chapter 57	Delivery Portfolio Discovery, Project Selection and Value Measurement
Chapter 58	Multi-Platform Pipeline Standardization Project
Chapter 59	Enterprise Kubernetes Delivery Platform Project
Chapter 60	Artifact Promotion, Release Trains and Environment Governance
Chapter 61	Brownfield Migration from Legacy Pipelines
Chapter 62	Pipeline Disaster Recovery, Business Continuity and Clean-Room Rebuild
Chapter 63	Advanced Jenkins Troubleshooting and Controller Incident Response
Chapter 64	GitHub Actions and GitLab CI/CD Incident Troubleshooting
Chapter 65	GitOps, Deployment, Identity and Supply-Chain Incident Response
Chapter 66	Delivery Performance, Scale, Reliability and Capacity Engineering
Chapter 67	Internal Developer Platform and Delivery Product Management
Chapter 68	CI/CD, GitOps, Kubernetes and DevSecOps Certification Mastery
Chapter 69	Technical Interviews and Delivery System Design Mastery
Chapter 70	Final Enterprise Software Delivery Factory Capstone
Part F Command and Investigation Reference	
Enterprise Delivery Graduation Checklist	
Ten Additional Portfolio Projects	
90-Day Enterprise Delivery Practice Plan	
Glossary	
Official Documentation Index	

Chapter 56: Enterprise CI/CD Architecture and Operating Model

Designing the delivery platform as a socio-technical production system.

Why this chapter matters: Enterprise delivery failures usually arise from unclear ownership, trust boundaries, shared dependencies, unsupported customization, and untested recovery—not from YAML syntax.

Learning Objectives

- Explain platform architecture and its business purpose.
- Design ownership and service model with explicit ownership and boundaries.
- Operate trust and failure boundaries using measurable service levels.
- Troubleshoot architecture evidence through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Platform architecture

Map repositories, CI controllers, runners, registries, artifact stores, identity providers, secret systems, policy engines, GitOps controllers, clusters, observability, evidence, and human decision points.

Ownership and service model

Treat delivery capabilities as products with consumers, owners, interfaces, SLOs, support, roadmap, versioning, deprecation, and recovery. Define RACI for source, pipeline, artifact, environment, runtime, security, and evidence.

Trust and failure boundaries

Separate untrusted validation, trusted build, release, signing, promotion, production deployment, and break-glass. Bound state, credentials, queues, plugins, runners, clusters, repositories, and tenants.

Architecture evidence

A credible review includes requirements, quality attributes, diagrams, ADRs, identity and data flows, failure modes, capacity, cost, telemetry, runbooks, and measured tests—not only a product inventory.

Architecture and Operating Decisions

- Begin with business outcomes and quality-attribute scenarios.
- Assign one authoritative owner for every mutable property.
- Design control-plane recovery before scaling consumers.
- Prefer standardized interfaces over shell glue between platforms.
- Review architecture after major traffic, compliance, dependency, incident, or operating-model changes.

Reference Configuration or Command Pattern

```
adr_id: ADR-056
decision: separate-trust-tier-runner-pools
context: {forks: allowed, production_deployments: protected}
controls: [ephemeral-workers, short-lived-identity, network-segmentation]
tradeoffs: {cost: higher, blast_radius: lower, operations: more-complex}
recovery: {rto_minutes: 60, evidence: [runner-image, identity-policy, drill-report]}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.

- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Teams bypass platform	Golden path is slower or incomplete	Measure friction and improve supported path	Adoption and bypass audit
One outage stops all delivery	Shared control plane lacks isolation	Partition queues, workers, credentials and environments	Failure-domain test
No one owns failed release	Responsibility ends at pipeline completion	Assign service verification and incident ownership	RACI and timeline
Review is only a diagram	Requirements and tests are absent	Attach SLOs, threat model, recovery and cost evidence	Prioritized risk register

Hands-on Lab 56: Run an Enterprise Delivery Architecture Review

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Define five measurable business and engineering outcomes.
2. Map all delivery components, data flows, identities, trust boundaries, approvals, evidence, and managed environments.
3. Assign owners, SLOs, RTO/RPO, capacity, cost, and recovery for each control-plane component.
4. Create ADRs for runner isolation, artifact promotion, GitOps, policy, and evidence architecture.
5. Run tabletop scenarios for repository compromise, CI outage, registry outage, and credential misuse.
6. Prioritize the top ten risks and assign remediation owners and dates.
7. Publish the architecture package and review trigger.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. Why call CI/CD a platform product?

Teams depend on its reliability, interfaces, support, security, and evolution; treating it as scripts hides operational obligations.

2. What is usually the highest-risk asset?

A control plane, credential broker, signing authority, or privileged runner because compromise can authorize broad downstream change.

3. How should boundaries be chosen?

By ownership, trust, lifecycle, privilege, change rate, availability, compliance, and failure blast radius.

4. What proves architecture is operable?

Observable SLOs, clear ownership, and successful deployment, rollback, failover, and rebuild exercises.

Chapter Summary

- Enterprise delivery includes people, process, identity, evidence, and recovery.
- Control-plane compromise has outsized impact.
- Boundaries follow trust and failure domains.
- Architecture decisions require measured validation.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 57: Delivery Portfolio Discovery, Project Selection and Value Measurement

Choosing platform work from operational evidence rather than tool enthusiasm.

Why this chapter matters: Automation programs fail when they optimize the number of pipelines instead of lead time, reliability, risk, toil, adoption, and business outcomes.

Learning Objectives

- Explain opportunity discovery and its business purpose.
- Design value and risk model with explicit ownership and boundaries.
- Operate project selection using measurable service levels.
- Troubleshoot outcome measurement through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Opportunity discovery

Use incident records, change failures, ticket queues, audit findings, queue time, environment lead time, toil surveys, release delays, capacity constraints, and developer research to identify repeated pain.

Value and risk model

Estimate time avoided, flow improvement, failure reduction, risk reduction, evidence quality, opportunity cost, blast radius, reversibility, implementation effort, maintenance, and migration cost.

Project selection

Prefer stable, repeated, high-volume, well-understood processes with named owners and test environments. Delay automation when the process is defective, inputs are unstable, or authority is unclear.

Outcome measurement

Define baseline, target, sampling method, attribution, adoption, quality guardrails, and review date before implementation. Retire or redesign products that do not produce durable benefit.

Architecture and Operating Decisions

- Score proposals consistently and expose uncertainty.
- Include support, upgrade, deprecation, migration, and exception costs.
- Prioritize platform capabilities that remove repeated team-local work.
- Use conservative value attribution and verify after rollout.
- Balance flow, stability, security, user experience, and cost metrics.

Reference Configuration or Command Pattern

```
opportunity_score =
  frequency * minutes_saved * users
  + change_failure_reduction
  + risk_reduction
  + evidence_improvement
  - implementation_effort
  - maintenance_cost
  - blast_radius_penalty

entry_criteria: [stable_process, owner, test_environment, rollback]
```


Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Many projects, little impact	Selection follows enthusiasm, not evidence	Rebuild portfolio from toil, incidents and flow data	Baseline dashboard
Savings cannot be verified	No pre-change baseline	Measure manual process and use conservative attribution	Before/after sample
Users keep manual process	Product misses cases or is difficult	Interview users and improve ergonomics/migration	Adoption funnel
Platform team is overloaded	Every request becomes custom work	Define product tiers and reusable interfaces	Work-type and support metrics

Hands-on Lab 57: Create an Evidence-Based Delivery Portfolio

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Collect at least ten delivery opportunities from incidents, tickets, pipeline timing, audit, security, and user interviews.
2. Define baseline and owner for each opportunity.
3. Score value, risk, frequency, feasibility, blast radius, support, and migration.
4. Select three projects and write measurable outcome hypotheses.
5. Reject or defer at least two proposals with rationale.
6. Design adoption, compatibility, support, deprecation, and post-launch review.
7. Publish a one-quarter roadmap with capacity and dependencies.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What makes a process suitable for automation?

It is stable, repeatable, understood, owned, measurable, testable, and reversible enough for the blast radius.

2. Why include adoption in value?

Unused automation produces maintenance cost without operational benefit.

3. What is a useful delivery baseline?

Current lead time, queue time, failure rate, recovery time, operator effort, risk, cost, and user satisfaction.

4. When should automation be retired?

When value, usage, compatibility, security, or maintainability no longer justifies continued support.

Chapter Summary

- Portfolio work begins with evidence.
- Value includes risk, maintenance and adoption.
- Unstable processes should be fixed before automated.
- Post-launch measurement determines continuation.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 58: Multi-Platform Pipeline Standardization Project

Creating consistent delivery contracts across Jenkins, GitHub Actions, and GitLab CI/CD.

Why this chapter matters: Enterprises often have several CI platforms; forcing one syntax everywhere or allowing unrestricted divergence both create migration, security, support, and evidence problems.

Learning Objectives

- Explain common delivery contract and its business purpose.
- Design reusable implementations with explicit ownership and boundaries.
- Operate identity and runner model using measurable service levels.
- Troubleshoot conformance and portability through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Common delivery contract

Standardize stages, inputs, artifact identity, test evidence, security controls, environment authorization, deployment verification, rollback, and release records independent of platform syntax.

Reusable implementations

Use Jenkins Shared Libraries, GitHub reusable workflows/actions, and GitLab CI/CD components/includes with pinned versions, schemas, compatibility, tests, documentation, and deprecation.

Identity and runner model

Map each platform's event, token, OIDC, secret, runner/agent, environment, approval, and audit semantics into common trust tiers without pretending they are identical.

Conformance and portability

Build contract tests that verify outputs and evidence. Portability means transferable behavior and data contracts, not lowest-common-denominator YAML or one large shell script.

Architecture and Operating Decisions

- Define vendor-neutral interfaces before implementations.
- Keep platform-specific strengths behind stable contracts.
- Version reusable components and provide upgrade automation.
- Test untrusted contribution, cancellation, retry, cache, secret, artifact, and environment behavior on every platform.
- Normalize release evidence by source revision and artifact digest.

Reference Configuration or Command Pattern

```
delivery_contract:
  inputs: [source_revision, build_parameters]
  outputs: [artifact_digest, test_report, sbom, provenance]
  gates: [review, policy, environment_authorization]
  deploy: {strategy: declared, verification: required}
  recovery: {rollback_or_fix_forward: documented}
  implementations: [jenkins-library-v3, github-workflow-v4, gitlab-component-v2]
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.

- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Platforms produce different artifacts	Contract leaves packaging and metadata ambiguous	Define canonical output and conformance tests	Matching digest/metadata
Reusable component update breaks teams	No versioning or compatibility policy	Pin versions, test matrix and staged upgrade	Consumer compatibility report
OIDC works on one platform only	Trust assumptions were copied literally	Model claims and authorization per platform	Negative claim tests
Standardization becomes shell framework	Logic is hidden from platform observability	Use native primitives behind common interfaces	Stage-level telemetry

Hands-on Lab 58: Implement One Delivery Contract on Three Platforms

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Define a service delivery contract covering source, build, test, artifact, security, promotion, deployment, verification, evidence, and recovery.
2. Implement it as a Jenkins Shared Library pipeline.
3. Implement it as a GitHub reusable workflow with protected environments and short-lived identity.
4. Implement it as a GitLab CI/CD component with scoped runners and deployments.
5. Run the same service revision and compare artifact and evidence outputs.
6. Test fork/untrusted events, cancellation, retry, missing secret, unavailable dependency, and failed deployment.
7. Publish a compatibility matrix, migration guide, and version policy.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What should be standardized first?

The delivery contract, evidence, trust boundaries, and expected outcomes—not the syntax.

2. Why not hide everything in a shell script?

It reduces platform-native visibility, cancellation, security, stage telemetry, and maintainability.

3. What is conformance testing?

Tests that prove each implementation satisfies the same behavioral and evidence contract.

4. How should reusable components evolve?

Semantic or documented versioning, compatibility tests, deprecation windows, migration tools, telemetry, and support.

Chapter Summary

- Standardize behavior and evidence.
- Use native platform capabilities behind contracts.
- Identity models require platform-specific policy.
- Reusable components are products with lifecycle.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 59: Enterprise Kubernetes Delivery Platform Project

Applying CI, GitOps, progressive delivery, policy, evidence, and recovery to multiple teams and clusters.

Why this chapter matters: A production Kubernetes platform must connect artifact creation to controlled desired state, reconciliation, health, progressive exposure, security, and incident recovery.

Learning Objectives

- Explain platform topology and its business purpose.
- Design tenant and environment model with explicit ownership and boundaries.
- Operate release flow using measurable service levels.
- Troubleshoot reliability and recovery through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Platform topology

Design source, build, registry, environment repositories, Argo CD or Flux, policy, secrets, progressive delivery, clusters, observability, evidence, and ownership as one system.

Tenant and environment model

Separate platform and application responsibilities using repositories, projects or Kustomizations, namespaces, clusters, service accounts, quotas, network boundaries, and support tiers.

Release flow

Build once, generate evidence, promote immutable digest, update desired state, reconcile, analyze, verify user behavior, and record release. Keep emergency changes traceable.

Reliability and recovery

Define controller SLOs, reconciliation age, cluster onboarding, backup, upgrade, failure isolation, clean bootstrap, drift response, rollback, and multi-cluster continuity.

Architecture and Operating Decisions

- Choose one primary GitOps controller and record the trade-off.
- Use least privilege and bounded controller scope.
- Verify rendered manifests and artifact identity before admission.
- Run progressive delivery only with trustworthy metrics and abort behavior.
- Test controller loss, stale source, bad configuration, unavailable cluster, and secret outage.

Reference Configuration or Command Pattern

```
release_flow:
  build: {artifact: sha256:<digest>, evidence: [tests, sbom, provenance, signature]}
  promote: {environment_repo_change: reviewed}
  reconcile: {controller: argocd-or-flux, policy: pass}
  progress: {strategy: canary, analysis: [latency, errors, business_success]}
  verify: {runtime_digest: match, user_path: pass}
  recover: {abort_or_rollback: tested}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.

- Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Controller says Synced but service fails	Operational verification is missing	Add health, smoke, dependency and SLO checks	User-path success
One tenant affects all clusters	Controller, credentials or quotas are shared broadly	Partition trust and capacity boundaries	Isolation game day
Canary passes on low traffic	Analysis lacks sample sufficiency	Pause or mark inconclusive until representative data	Sample threshold evidence
Controller rebuild loses access	Bootstrap credentials or registrations are not recoverable	Protect inputs and rehearse clean bootstrap	New-cluster recovery

Hands-on Lab 59: Deliver a Multi-Team Kubernetes Release Platform

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Define two teams, two workload classes, three environments, trust boundaries, SLOs, RTO/RPO, and cost limits.
2. Build a reusable Helm or Kustomize contract and immutable artifact promotion.
3. Deploy Argo CD or Flux with tenant projects/service accounts, policy, secret references, and notifications.
4. Implement rolling and canary release paths with business and technical validation.
5. Onboard two applications and measure lead time and support friction.
6. Run drift, invalid desired state, policy outage, controller loss, and unavailable cluster scenarios.
7. Publish architecture, runbooks, dashboards, release records, and recovery evidence.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What is the deployment source of truth?

Versioned desired state plus the immutable artifact identity it references; runtime health remains authoritative for service behavior.

2. How should multi-tenancy be designed?

From trust, compliance, blast radius, capacity, ownership, and recovery—not namespace count alone.

3. Why build once and promote?

It preserves the exact artifact that passed tests and evidence checks.

4. What proves GitOps platform resilience?

Other tenants continue during a failure and the controller/cluster can be restored within objectives from protected sources.

Chapter Summary

- Kubernetes delivery is an integrated platform.
- Tenant boundaries combine identity, repositories, controllers and clusters.
- Promotion preserves artifact identity.
- Recovery must include GitOps control planes.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 60: Artifact Promotion, Release Trains and Environment Governance

Coordinating immutable releases, dependencies, approvals, calendars, and emergency change.

Why this chapter matters: Large organizations need both independent team flow and coordinated risk control; copying builds, manual environment changes, and opaque release calendars undermine both.

Learning Objectives

- Explain artifact promotion and its business purpose.
- Design release trains with explicit ownership and boundaries.
- Operate environment governance using measurable service levels.
- Troubleshoot emergency release through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Artifact promotion

Move or reference the same immutable digest across environments with SBOM, provenance, signature, test, vulnerability, policy, and approval evidence. Separate build authorization from deployment authorization.

Release trains

Coordinate dependent services, shared infrastructure, data changes, regulatory windows, customer communications, freeze periods, and rollback ownership. Use trains where coupling is real, not as a universal batch process.

Environment governance

Define ownership, access, configuration, data, quotas, change windows, validation, retention, drift, cost, refresh, and deletion for every environment. Ephemeral and long-lived environments need different controls.

Emergency release

Use a pre-authorized, strongly authenticated, least-privilege path with incident/change record, independent review where possible, exact artifact identity, validation, communication, and post-event reconciliation.

Architecture and Operating Decisions

- Never rebuild an approved release for the next environment.
- Model cross-service and data compatibility explicitly.
- Use risk-based approvals and automated evidence rather than ceremony alone.
- Make environment state observable and reproducible.
- Test emergency and rollback paths outside an emergency.

Reference Configuration or Command Pattern

```
promotion_record:
  artifact_digest: sha256:<digest>
  from: staging
  to: production
  evidence: [tests, sbom, provenance, signature, policy, staging_slo]
  dependencies: [orders-api>=4.2, schema>=17]
  approver: <environment-owner>
  window: <change-window>
  rollback: <previous-digest>
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Production digest differs from staging	Artifact was rebuilt or tag moved	Quarantine release and promote immutable digest	Digest equality
Train waits indefinitely	Dependencies and entry criteria are unclear	Define ownership, readiness and decoupling plan	Dependency board
Environment has unknown changes	Manual access and drift are uncontrolled	Inventory, reconcile and restrict changes	Environment diff
Emergency release bypasses evidence	Break-glass path is untested or too broad	Use prepared minimum evidence and retrospective review	Complete emergency record

Hands-on Lab 60: Operate a Governed Release Train

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Select three sample services with one schema and one API dependency.
2. Define compatibility, train entry/exit, owners, schedule, communications, capacity, and rollback.
3. Build immutable artifacts and gather promotion evidence.
4. Promote through staging to production without rebuilding.
5. Introduce one delayed dependency and decide whether to decouple, defer, or stop.
6. Run an emergency release using the controlled path.
7. Measure lead time, waiting, failure, rollback, and train value.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. When is a release train justified?

When real dependency, compliance, communication, or operational coupling requires coordinated change.

2. Why separate build and deploy authorization?

Producing bytes and exposing them to a protected environment are different risk decisions.

3. What is environment drift?

Unreviewed difference between declared or expected configuration and observed state.

4. What makes emergency release safe enough?

Predefined scope, strong identity, minimum evidence, independent oversight, verification, communication, and reconciliation.

Chapter Summary

- Promote immutable artifacts with evidence.
- Coordinate only real dependencies.
- Environments are governed products.
- Emergency paths need rehearsal and reconciliation.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 61: Brownfield Migration from Legacy Pipelines

Discovering, rationalizing, converting, validating, and retiring unsafe or unsupported delivery paths.

Why this chapter matters: Legacy pipelines encode hidden business logic, credentials, manual approvals, environment assumptions, and recovery knowledge; a syntax conversion alone can lose critical behavior.

Learning Objectives

- Explain discovery and inventory and its business purpose.
- Design rationalization with explicit ownership and boundaries.
- Operate migration waves using measurable service levels.
- Troubleshoot behavioral equivalence through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Discovery and inventory

Collect repositories, jobs, triggers, scripts, plugins, agents, credentials, artifacts, environments, owners, schedules, dependencies, manual steps, failure rates, support status, and business criticality.

Rationalization

Retain, retire, replace, refactor, rehost, or consolidate each pipeline. Remove dead paths before migration and separate application-specific logic from reusable platform capability.

Migration waves

Group by risk, dependency, owner readiness, platform compatibility, and rollback. Define dual-run or shadow comparison, entry/exit, freeze, cutover, validation, and retirement.

Behavioral equivalence

Compare source revision, parameters, dependencies, artifact digest or semantics, tests, security, approvals, deployment behavior, evidence, performance, and failure handling. Improvement is allowed but must be intentional and measured.

Architecture and Operating Decisions

- Do not migrate unknown credentials or plugins blindly.
- Create a canonical pipeline model and automated discovery where possible.
- Prioritize unsupported, high-risk, high-toil paths with available owners.
- Keep rollback to the legacy path time-bounded and secure.
- Delete old triggers, credentials, agents, webhooks, artifacts, and access after acceptance.

Reference Configuration or Command Pattern

```
migration_inventory:
  pipeline_id: legacy-payments-prod
  owner: payments-team
  source: scm-job
  credentials: [registry-write, deploy-prod]
  plugins: [<inventory>]
  artifact: image
  environments: [staging, prod]
  hidden_steps: [manual-db-check]
  decision: refactor-to-golden-path
  wave: 2
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.

- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Migrated build produces different result	Hidden script, plugin or environment input	Compare full inputs and artifact contents	Equivalence report
Both pipelines deploy at once	Triggers were not isolated during dual run	Disable deployment in shadow path and add idempotency	Single deployment audit
Owner refuses cutover	Critical behavior or support is missing	Close gaps and rehearse rollback with owner	Acceptance sign-off
Legacy system remains privileged	Retirement criteria omit credentials and agents	Revoke and verify no use after cutover	Retirement audit

Hands-on Lab 61: Build a Pipeline Migration Factory

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Inventory at least five representative legacy pipelines.
2. Extract canonical triggers, inputs, steps, credentials, artifacts, deployments, approvals, evidence, and failure behavior.
3. Classify each as retain, retire, replace, refactor, rehost, or consolidate.
4. Create a golden-path target and automated conversion for common stages.
5. Run one pipeline in shadow mode and compare behavior and outputs.
6. Cut over with rollback criteria and validate production behavior.
7. Retire old jobs, agents, credentials, triggers, artifacts, and documentation.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. Why is pipeline migration not file conversion?

Pipelines encode behavior, permissions, dependencies, manual knowledge, artifacts, and operational recovery beyond syntax.

2. What is shadow mode?

The new path runs for comparison without performing unsafe side effects such as production deployment.

3. What should determine waves?

Risk, business criticality, dependencies, owner readiness, compatibility, testability, and rollback.

4. When is migration complete?

The target is accepted and the old execution authority, credentials, triggers, agents, and support burden are removed.

Chapter Summary

- Discovery reveals hidden delivery behavior.
- Rationalize before migration.
- Validate behavioral equivalence and intentional improvements.
- Retirement includes authority and infrastructure.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 62: Pipeline Disaster Recovery, Business Continuity and Clean-Room Rebuild

Restoring trusted delivery when controllers, repositories, registries, identities, or evidence systems fail.

Why this chapter matters: Application recovery depends on delivery capability; a platform that cannot release, roll back, or rebuild during an incident is itself a critical business dependency.

Learning Objectives

- Explain continuity requirements and its business purpose.
- Design backup and rebuild with explicit ownership and boundaries.
- Operate degraded operation using measurable service levels.
- Troubleshoot clean-room recovery through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Continuity requirements

Define delivery RTO/RPO by platform capability: source, controller, runners, registry, secrets, identity, policy, GitOps, evidence, DNS, network, and communications. Identify minimum viable delivery during crisis.

Backup and rebuild

Back up only necessary state and protect encryption keys, credentials, configuration, plugins, job history, artifact metadata, registrations, and audit evidence. Prefer reproducible controller and runner rebuild from versioned configuration.

Degraded operation

Document alternate runners, secondary registry, cached dependencies, manual approvals, emergency deployment, paused automation, and read-only operation. Degraded paths must preserve minimum identity, evidence, and rollback controls.

Clean-room recovery

When compromise is possible, rebuild from separately trusted source, images, identities, dependencies, and infrastructure. Exclude suspect caches, workers, keys, artifacts, and backups until validated.

Architecture and Operating Decisions

- Classify dependencies by recovery criticality and failure correlation.
- Test restoration, not only backup creation.
- Keep alternate authority and communications independent from the primary control plane.
- Define trust-restoration criteria separately from service restoration.
- Measure actual RTO/RPO and maintain remediation backlog.

Reference Configuration or Command Pattern

```
delivery_recovery_tiers:
  tier0: [identity, source, artifact-registry, emergency-communications]
  tier1: [ci-controller, runners, secret-broker, policy]
  tier2: [gitops, evidence-index, analytics]
minimum_viable_delivery:
  controls: [strong-identity, immutable-artifact, peer-approval, verification, audit]
clean_room: {new-identities: true, old-cache: forbidden}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.

- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Backup restores but jobs fail	Plugins, keys, credentials or external registrations are missing	Test full dependency restoration	Successful protected build
Secondary registry is stale	Replication RPO was not measured	Reconcile by digest and test promotion	Inventory consistency
DR uses same identity outage	Recovery path shares primary dependency	Create independent break-glass authority	Identity outage exercise
Service restored from suspect builder	Continuity confused with trust recovery	Quarantine and clean-room rebuild	New provenance and verification

Hands-on Lab 62: Recover the Delivery Platform from a Regional or Control-Plane Failure

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Map delivery dependencies, tiers, RTO/RPO, data/state, credentials, ownership, and failure correlation.
2. Create versioned controller, runner, policy, GitOps, and evidence configuration.
3. Back up required state and test restore into an isolated environment.
4. Simulate primary controller or region loss and activate minimum viable delivery.
5. Verify one safe build, promotion, deployment, and rollback through the recovery path.
6. Repeat as a compromise scenario using clean-room identities and no old cache.
7. Record actual recovery, gaps, communications, cost, and remediation.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. Why is CI/CD part of business continuity?

Teams need trusted build, deploy, rollback, and repair capability during application or infrastructure incidents.

2. What is minimum viable delivery?

The smallest protected path that can release or recover critical services with identity, immutable artifacts, approval, verification, and audit.

3. Why can a backup be unsafe?

It may contain compromised configuration, credentials, plugins, artifacts, or encrypted secrets whose keys are also affected.

4. What proves DR readiness?

Measured end-to-end restoration and trusted release within objectives, including people, dependencies, and communications.

Chapter Summary

- Delivery continuity is a production requirement.
- Reproducible rebuild complements backup.
- Degraded operation preserves minimum controls.
- Clean-room trust recovery differs from ordinary restore.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 63: Advanced Jenkins Troubleshooting and Controller Incident Response

Diagnosing queue, controller, agent, plugin, pipeline, credential, and storage failures safely.

Why this chapter matters: Jenkins is extensible and stateful; controller overload, plugin interactions, persistent workspaces, shared libraries, credentials, or storage problems can create broad delivery outages.

Learning Objectives

- Explain evidence-first triage and its business purpose.
- Design controller and queue with explicit ownership and boundaries.
- Operate agents and workspaces using measurable service levels.
- Troubleshoot plugin and recovery discipline through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Evidence-first triage

Establish impact, scope, recent change, controller health, queue, executors, agents, logs, thread dumps, heap, disk, I/O, GC, plugin status, credentials, SCM, and dependency health before changing the system.

Controller and queue

Slow UI, queue growth, blocked executors, CPS persistence, excessive logs, large artifacts, controller-side shell steps, and inefficient shared libraries can saturate CPU, memory, storage, or serialization.

Agents and workspaces

Labels, capacity, connectivity, Java/runtime compatibility, remoting, disk, container lifecycle, cache, tool installation, permissions, and stale workspace state cause failures that appear as pipeline defects.

Plugin and recovery discipline

Inventory versions and dependencies, test upgrades, use support bundles carefully, back up JENKINS_HOME and secrets appropriately, avoid random restarts, and document rollback. A compromised controller may require clean rebuild, not restoration.

Architecture and Operating Decisions

- Mitigate user impact before deep optimization.
- Collect thread/heap/support evidence before restart when safe.
- Do not expose credentials in support bundles or console output.
- Change one hypothesis at a time and verify.
- Separate availability restoration from compromise investigation.

Reference Configuration or Command Pattern

```
incident_hypothesis:
  symptom: queue_wait_p95_gt_15m
  scope: all-linux-builds
  recent_change: agent-image-v42
  evidence: [queue, node-offline-cause, remoting-log, disk-metrics]
  safe_test: launch-clean-agent-v41
  mitigation: restore-v41-pool
  validation: queue_wait_p95_lt_2m
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.

- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Queue grows with idle agents	Label, cloud limit, executor, node property or provisioning failure	Inspect blockage reason and provisioning logs	Jobs scheduled on intended pool
Controller CPU is high	Controller-side work, plugin loop or CPS/log load	Capture threads, reduce workload and isolate culprit	CPU and latency recover
Pipeline resumes incorrectly	Nonserializable state or plugin/library incompatibility	Use safe pipeline patterns and compatible versions	Restart/replay test
Credentials appear in logs	Shell expansion, debug or masking gap	Revoke, restrict, fix invocation and inspect exposure	Rotation and negative test

Hands-on Lab 63: Run a Jenkins Controller and Agent Incident Drill

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Create a sandbox Jenkins controller with a shared library and two agent pools.
2. Establish baseline queue, stage, CPU, memory, storage, GC, agent, and build metrics.
3. Inject a label mismatch, offline agent, disk pressure, and slow controller-side step separately.
4. For each incident, record impact, evidence, hypotheses, safe test, mitigation, recovery, and prevention.
5. Capture a secret-safe support bundle, thread dump, and relevant logs.
6. Test plugin/shared-library rollback and controller restore in isolation.
7. Publish incident runbooks and SLO improvements.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What should happen before restarting Jenkins?

Preserve relevant evidence such as logs, queue state, threads, memory or storage data when safe and assess interrupted builds.

2. Why avoid builds on the controller?

Untrusted or resource-intensive work can compromise or destabilize the control plane.

3. What commonly causes queued jobs with idle executors?

Label or node-property mismatch, provisioning limits, permissions, locks, throttling, or jobs reserved for another node class.

4. When is clean rebuild preferable?

After suspected controller compromise or when state, plugins, credentials, and provenance cannot be trusted.

Chapter Summary

- Jenkins troubleshooting begins with impact and evidence.
- Controller, queue, agents, plugins and storage interact.
- One hypothesis and safe test at a time improves accuracy.
- Compromise recovery may require clean rebuild.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 64: GitHub Actions and GitLab CI/CD Incident Troubleshooting

Resolving workflow, runner, token, cache, artifact, environment, and platform failures.

Why this chapter matters: SaaS CI failures often span event semantics, YAML evaluation, reusable components, permissions, runner networks, OIDC claims, rate limits, artifacts, caches, environments, and external dependencies.

Learning Objectives

- Explain run and event analysis and its business purpose.
- Design runner and dependency path with explicit ownership and boundaries.
- Operate identity and environments using measurable service levels.
- Troubleshoot platform incident response through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Run and event analysis

Identify event type, source revision, workflow/config revision, actor, permissions, inputs, concurrency, cancellation, reusable call chain, matrix expansion, environment, and deployment record.

Runner and dependency path

Inspect hosted versus self-hosted runners, labels/tags, executor image, network, DNS, proxy, storage, container pulls, service containers, cache, artifact, package registry, API limits, and external status.

Identity and environments

Tokens and OIDC claims differ by event, branch, repository/project, workflow/job, environment, and fork trust. Protected environments, approvals, deployment tiers, and job tokens can block or over-authorize execution.

Platform incident response

Distinguish configuration defect, customer runner issue, external dependency failure, and provider outage. Preserve logs and evidence, use degraded paths carefully, and reconcile reruns so duplicate side effects do not occur.

Architecture and Operating Decisions

- Inspect exact resolved configuration and commit.
- Never rerun deployment jobs blindly without idempotency analysis.
- Validate OIDC claim and cloud trust policy together.
- Treat caches and artifacts as untrusted inputs across trust tiers.
- Use provider status as one signal, not a substitute for local evidence.

Reference Configuration or Command Pattern

```
triage:
  platform: github-or-gitlab
  run_id: <id>
  event: pull_request-or-push
  source_revision: <sha>
  config_revision: <sha>
  runner: <image-and-labels>
  identity_claims: <sanitized>
  failed_boundary: auth/network/cache/artifact/environment
  rerun_safe: false-until-proven
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Workflow not triggered	Path/branch rules, event semantics or config location	Inspect event payload and default-branch config rules	Expected trigger test
Cloud login denied	OIDC audience/subject or environment claim mismatch	Compare job claims with trust policy	Accepted and denied claims
Self-hosted runner accepts unsafe job	Runner group/tag scope is broad	Restrict repository/project and trust-tier routing	Fork negative test
Rerun duplicates deployment	Job is not idempotent and prior result is unknown	Determine external state and use reconciliation	Single final deployment

Hands-on Lab 64: Investigate a Cross-Platform CI Incident Set

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Create equivalent GitHub Actions and GitLab pipelines using self-hosted or simulated runner constraints.
2. Establish evidence fields for event, source, config, actor, token, runner, artifact, environment, and deployment.
3. Inject trigger-filter, OIDC-claim, runner-network, cache, artifact-retention, and protected-environment failures.
4. Diagnose each using logs, resolved configuration, audit/deployment records, and external state.
5. Decide whether rerun, retry, rollback, cancel, or reconcile is safe.
6. Simulate a provider dependency outage and execute degraded-operation policy.
7. Publish a comparative troubleshooting matrix and prevention tests.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. Why can the same workflow behave differently on pull request and push?

Event context, token permissions, secret access, source checkout, fork trust, branch protections, and environment rules differ.

2. What should be checked for OIDC failure?

Issuer, audience, subject and other claims from the job plus the cloud trust policy and role permissions.

3. Why are reruns risky?

The first attempt may have produced external side effects even if the platform reports failure or lost status.

4. How should provider outages be handled?

Confirm scope, protect in-flight changes, use approved degraded paths, communicate, and reconcile state after recovery.

Chapter Summary

- SaaS CI incidents span events, runners, identity and dependencies.
- Exact source and configuration revisions matter.
- Reruns require side-effect analysis.
- Cross-platform principles remain transferable.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 65: GitOps, Deployment, Identity and Supply-Chain Incident Response

Coordinating failures that cross repository, artifact, policy, controller, cluster, and runtime boundaries.

Why this chapter matters: The hardest delivery incidents do not remain inside one tool: a compromised identity, wrong artifact, stale desired state, policy outage, or controller mutation can propagate across the entire release path.

Learning Objectives

- Explain incident command and its business purpose.
- Design cross-boundary diagnosis with explicit ownership and boundaries.
- Operate containment and recovery using measurable service levels.
- Troubleshoot post-incident improvement through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Incident command

Define incident commander, operations, investigation, communications, security, service owner, evidence custodian, and decision authority. Maintain one timeline and explicit objectives.

Cross-boundary diagnosis

Correlate source and workflow revision, actor, runner, artifact digest, SBOM/provenance/signature, promotion, desired state, controller decision, policy, cluster events, runtime digest, telemetry, and customer impact.

Containment and recovery

Pause unsafe reconciliation or release, revoke identities, quarantine artifacts, restore stable traffic, preserve evidence, narrow blast radius, and choose rollback, fix-forward, clean rebuild, or environment restore based on trust.

Post-incident improvement

Distinguish trigger, root, contributing conditions, detection gaps, failed controls, recovery friction, and organizational causes. Assign measurable actions, owners, dates, and retests.

Architecture and Operating Decisions

- Optimize first for user safety and evidence preservation.
- Do not destroy potentially useful evidence during containment.
- Verify runtime artifact and configuration rather than trusting dashboards alone.
- State uncertainty and affected scope honestly.
- Retest every high-priority corrective action.

Reference Configuration or Command Pattern

```
timeline_event:
  timestamp_utc: 2026-08-01T00:00:00Z
  actor: incident-commander
  observation: production digest differs from approved release
  evidence: [registry-log, deployment-record, pod-image-id]
  action: pause-promotion-and-gitops-sync
  result: no-new-exposure
  next_decision: scope-signing-identity-and-runner
```


Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Wrong artifact is running	Tag mutation, promotion error or admission bypass	Stop exposure, resolve digest chain and restore approved artifact	Runtime digest match
Controller reintroduces bad state	Desired state still contains incident change	Revert or pin known revision before resuming reconciliation	Stable repeated sync
Identity revoked but jobs continue	Cached token/session or alternate credential	Disable integrations, inspect active jobs and rotate dependent access	No successful old identity use
Teams disagree on scope	Evidence is fragmented and no commander owns decisions	Centralize timeline and affected-set hypothesis	Agreed scoped report

Hands-on Lab 65: Run an End-to-End Delivery Incident Game Day

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Define a synthetic scenario combining wrong artifact, excessive identity, and GitOps reconciliation.
2. Assign incident roles, authority, communications, evidence, stop conditions, and business objectives.
3. Detect customer impact and trace source-to-runtime identity.
4. Pause unsafe paths, revoke credentials, quarantine artifact, and restore stable service.
5. Decide whether ordinary rollback or clean-room rebuild is required.
6. Reconcile desired state, verify policy and runtime digest, and monitor recovery.
7. Conduct a blameless review and retest corrective actions.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What is the first goal in a delivery incident?

Protect users and business while preserving enough evidence to make safe trust and recovery decisions.

2. Why can rollback be unsafe?

The previous artifact, builder, identity, data state, or control plane may also be affected or incompatible.

3. What is an affected-set hypothesis?

A testable description of which source, builds, artifacts, identities, environments, and time window may be compromised.

4. What makes a post-incident action strong?

It has an owner, deadline, measurable expected outcome, validation method, and retest evidence.

Chapter Summary

- Delivery incidents cross tool boundaries.
- Correlate immutable identities and decisions.
- Containment protects users and trust.
- Corrective actions require retesting.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 66: Delivery Performance, Scale, Reliability and Capacity Engineering

Engineering feedback speed and release throughput without sacrificing stability or cost.

Why this chapter matters: Average pipeline duration hides queueing, long-tail variance, cache misses, runner saturation, external limits, noisy neighbors, and recovery backlog.

Learning Objectives

- Explain workload and queue model and its business purpose.
- Design performance decomposition with explicit ownership and boundaries.
- Operate capacity and reliability using measurable service levels.
- Troubleshoot economics and quality through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Workload and queue model

Measure arrivals by event and time, job classes, service times, concurrency, queue percentiles, cancellation, retry, priority, fairness, and burst behavior. Separate compute demand from external dependency waiting.

Performance decomposition

Break lead time into review, queue, checkout, dependency, build, test, scan, artifact, approval, deployment, reconciliation, verification, and recovery. Optimize the material constraint rather than every stage.

Capacity and reliability

Include baseline, peak, growth, failure reserve, deployment surge, autoscaling lag, cold start, quota, regional failure, maintenance, and recovery backlog. Define graceful degradation and load shedding.

Economics and quality

Track cost per build/release, cache effectiveness, waste, reserved versus elastic capacity, failure cost, developer time, and environmental footprint. Faster is valuable only if correctness, security, and stability remain acceptable.

Architecture and Operating Decisions

- Set SLOs by job and risk class.
- Use representative load, burst, soak, dependency-failure, and failover tests.
- Protect high-priority recovery and release jobs from bulk workloads.
- Treat cache as an integrity and cost boundary.
- Plan quotas and alternate capacity before major events.

Reference Configuration or Command Pattern

```
delivery_slos:
  pr-feedback-p95: 10m
  queue-wait-p95: 90s
  release-p95: 25m
  deployment-success: 99.5%
  platform-availability: 99.9%
  recovery-rto: 60m
capacity:
  peak_concurrency: 180
  failure_reserve: 30%
  recovery_priority_pool: dedicated
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
More runners do not reduce lead time	Constraint is tests, registry, dependency or approval	Use stage and queue decomposition	Reduced bottleneck time
Autoscaling oscillates	Signal, cooldown or job length mismatched	Tune on queue age and class; set bounds	Stable queue and utilization
Cache improves speed but causes failures	Key or trust scope is too broad	Version cache inputs and separate trust tiers	Reproducible clean-cache build
Regional failure exhausts capacity	No reserve or quota in recovery location	Pre-provision quota and prioritize critical jobs	Failover load test

Hands-on Lab 66: Build a Delivery Capacity and Reliability Model

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Collect two weeks or simulate representative job arrivals, classes, service times, queue, failures, retries, and costs.
2. Create stage decomposition and identify the primary constraint.
3. Define SLOs and capacity for normal, peak, maintenance, and regional-failure conditions.
4. Run baseline, burst, soak, dependency-latency, and runner-loss tests.
5. Tune concurrency, autoscaling, caching, priority, and quotas one change at a time.
6. Measure cost per successful build and release plus quality guardrails.
7. Publish a capacity forecast, runbook, and investment backlog.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. Why use percentiles instead of averages?

Long-tail queue and stage times determine developer and release experience and are hidden by averages.

2. What limits delivery throughput?

The narrowest combined constraint across runners, tests, dependencies, registries, quotas, approvals, deployment, and recovery.

3. How should recovery jobs be prioritized?

Reserve capacity and explicit priority so bulk validation cannot block critical repair or rollback.

4. When is a cache unsafe?

When untrusted jobs can poison entries used by trusted builds or keys omit relevant inputs.

Chapter Summary

- Model arrivals, queues, stages and dependencies.
- Capacity includes failure reserve and recovery.
- Optimize the constraint, not every metric.
- Performance, quality, security and cost are coupled.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 67: Internal Developer Platform and Delivery Product Management

Turning pipelines, environments, GitOps, security, and evidence into supported developer products.

Why this chapter matters: Platform engineering succeeds through adoption, trust, usability, reliability, and measurable outcomes—not through the number of templates or portals delivered.

Learning Objectives

- Explain product discovery and its business purpose.
- Design golden paths and interfaces with explicit ownership and boundaries.
- Operate platform slo and support using measurable service levels.
- Troubleshoot adoption and lifecycle through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Product discovery

Identify developer jobs, pain, cognitive load, team differences, compliance needs, service ownership, and existing workarounds through interviews, journey mapping, support data, telemetry, and incidents.

Golden paths and interfaces

Provide service templates, pipeline components, environment vending, artifact/promotion, GitOps, observability, policy, documentation, APIs/CLI/portal, scorecards, and support. Keep underlying systems accessible for expert diagnosis.

Platform SLO and support

Define availability, request lead time, queue, documentation freshness, support response, change communication, onboarding, compatibility, and recovery. Publish status and ownership.

Adoption and lifecycle

Measure discovery, activation, successful first deployment, repeated use, upgrade coverage, bypass, satisfaction, support burden, and outcome improvement. Version products, manage deprecation, and remove obsolete paths.

Architecture and Operating Decisions

- Segment users and workload classes rather than building one rigid path.
- Make safe defaults fast and override behavior explicit.
- Treat documentation, examples, migration and support as product features.
- Avoid mandatory adoption before the platform handles required cases.
- Use outcome metrics and user research to prioritize roadmap.

Reference Configuration or Command Pattern

```
platform_product:
  capability: secure-service-delivery
  consumers: [api-teams, workers, scheduled-jobs]
  interfaces: [template, cli, portal, api]
  slos: {onboarding_p90: 1d, deploy_success: 99.5%, support_response: 4h}
  adoption: {activated_teams: 42, bypass_rate: 6%}
  lifecycle: {current: v4, deprecates: v2, migration_tool: available}
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.

- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Portal adoption is low	It adds steps without solving user jobs	Research workflow and improve end-to-end path	Activation and repeat use
Golden path becomes restrictive	Risk tiers and extension interfaces are absent	Add supported extension points and governed custom path	Reduced bypass
Teams remain on old version	Upgrade cost and compatibility are unclear	Provide automated migration and telemetry	Version coverage
Platform hides outages	No published SLO or status	Instrument service and communicate incidents	Trust and support metrics

Hands-on Lab 67: Design and Validate a Delivery Platform Product

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Interview at least three representative developer or simulated personas.
2. Map the journey from repository creation through production operation and incident response.
3. Define two golden paths, one extension model, service catalog, ownership, SLOs, support, and documentation.
4. Implement a minimal self-service interface that creates repository, pipeline, artifact, environment, GitOps, policy, and telemetry scaffolding.
5. Onboard two users and observe completion without coaching.
6. Measure activation, time, errors, support, bypass, and satisfaction.
7. Prioritize improvements and publish version/deprecation roadmap.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What is the customer of an internal platform?

Internal product and engineering teams that need reliable capabilities to deliver and operate services.

2. What makes a golden path good?

It is easy, safe, observable, supported, extensible enough, and aligned with common workload needs.

3. Why measure bypass?

Bypass reveals missing capabilities, friction, trust issues, or policy conflicts.

4. Should a platform hide all complexity?

It should reduce unnecessary cognitive load while preserving debuggability and expert escape paths.

Chapter Summary

- Platform engineering is product management plus engineering.
- Golden paths require user research and support.
- SLOs and status build trust.
- Lifecycle and adoption determine durable value.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 68: CI/CD, GitOps, Kubernetes and DevSecOps Certification Mastery

Using certifications to structure learning while proving skill through labs and portfolio evidence.

Why this chapter matters: Credentials can validate a knowledge domain, but memorized questions do not demonstrate operating judgment, troubleshooting, recovery, or current platform competence.

Learning Objectives

- Explain role-aligned selection and its business purpose.
- Design study system with explicit ownership and boundaries.
- Operate hands-on preparation using measurable service levels.
- Troubleshoot evidence and ethics through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Role-aligned selection

Choose credentials from target roles and gaps: Kubernetes administration/development/security, GitHub or GitLab platform operation, cloud DevOps, security, or vendor ecosystem. Verify current official objectives and prerequisites.

Study system

Map domains to concepts, commands, labs, failure modes, evidence, and review cadence. Use active recall, spaced repetition, timed practice, documentation navigation, and error logs.

Hands-on preparation

Rebuild labs from memory, explain decisions, troubleshoot deliberately broken systems, work under time limits, and practice cleanup. Certification preparation should strengthen real operational skill.

Evidence and ethics

Use official guides and legitimate practice. Do not use leaked exam content. Pair credentials with repositories, diagrams, runbooks, incident reports, measurable results, and honest scope statements.

Architecture and Operating Decisions

- Verify exam version, delivery environment, renewal, and allowed resources before registration.
- Create a gap matrix rather than rereading everything equally.
- Spend more time on weak domains and integrated scenarios.
- Practice terminal speed only after conceptual correctness.
- Treat a failed practice attempt as diagnostic evidence.

Reference Configuration or Command Pattern

```
certification_gap_matrix:
  domain: kubernetes-troubleshooting
  target_weight: verify-current-guide
  confidence: 2/5
  evidence: [failed-network-policy-lab, slow-log-navigation]
  actions: [repeat-lab-from-memory, timed-triage, command-drill]
  review_date: 2026-08-15
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.

- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Practice scores plateau	Review is passive and errors are not classified	Create error log and targeted labs	Weak-domain improvement
Lab works only from notes	Recall and diagnosis are not practiced	Rebuild from blank environment and explain aloud	Independent completion
Candidate studies outdated blueprint	Unofficial material or old guide	Use current official objectives and docs	Updated gap matrix
Credential does not help interview	No portfolio stories or operational depth	Pair domains with projects and incidents	Evidence-backed explanations

Hands-on Lab 68: Build a Role-Based Certification Preparation System

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Choose one target role and two relevant current certifications or skill assessments.
2. Retrieve official objectives and map each domain to existing Volume 10 chapters.
3. Score confidence and evidence for every domain.
4. Create a six-week plan using active recall, labs, troubleshooting, timed practice, and review.
5. Complete one full practical mock and classify every error.
6. Build portfolio artifacts for the three weakest domains.
7. Run a final readiness review based on evidence rather than confidence alone.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. Is certification enough for a DevOps role?

No. It validates a bounded knowledge domain and should be paired with hands-on projects, troubleshooting, communication, and portfolio evidence.

2. How should study time be allocated?

By objective weight, confidence, error history, operational importance, and available evidence.

3. Why practice failures?

Production and practical exams require diagnosis and recovery, not only happy-path recall.

4. What is unethical exam preparation?

Using leaked questions, copied answers, unauthorized assistance, or material that violates exam rules.

Chapter Summary

- Choose credentials by role and gaps.
- Use official current objectives.
- Active labs and error analysis outperform passive review.
- Portfolio evidence gives credentials practical meaning.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 69: Technical Interviews and Delivery System Design Mastery

Communicating architecture, troubleshooting, security, operations, and trade-offs under pressure.

Why this chapter matters: Strong candidates make assumptions visible, structure ambiguous problems, reason from evidence, and connect tools to operating outcomes instead of reciting product features.

Learning Objectives

- Explain answer structures and its business purpose.
- Design system-design flow with explicit ownership and boundaries.
- Operate depth and honesty using measurable service levels.
- Troubleshoot story bank and portfolio through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Answer structures

Use STAR plus learning for behavioral stories; impact, scope, evidence, hypotheses, safe tests, mitigation, recovery, and prevention for incidents; goals, constraints, scale, boundaries, design, critical paths, failure, security, operations, cost, and trade-offs for architecture.

System-design flow

Clarify users, repositories, event rate, languages, artifact types, environments, deployment frequency, latency, compliance, availability, RTO/RPO, tenancy, budget, team skills, and migration constraints before naming products.

Depth and honesty

Explain source-to-runtime identity, retries, cancellation, caches, artifacts, promotion, runners, credentials, quotas, failure modes, observability, incident command, and recovery. State unknowns and verification method instead of bluffing.

Story bank and portfolio

Prepare stories for migration, outage, security improvement, cost reduction, performance bottleneck, disagreement, failed experiment, platform adoption, mentoring, and architecture trade-off. Quantify impact and isolate your contribution.

Architecture and Operating Decisions

- Draw trust and failure boundaries before detailed components.
- Name at least one credible alternative and explain why it lost.
- Include operations and recovery in every design.
- Revise the design when the interviewer changes a constraint.
- Sanitize employer and customer details in stories.

Reference Configuration or Command Pattern

```
system_design_outline:
1: clarify outcomes and constraints
2: estimate scale and critical paths
3: draw trust, data and failure boundaries
4: design source, build, artifact, deploy and operate paths
5: cover security, SLOs, DR, cost and migration
6: compare alternatives and residual risk
7: summarize evidence and next validation
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Answer becomes tool list	Requirements and boundaries were skipped	Restart with outcomes and critical paths	Choices map to requirements
Candidate overclaims	Uncertainty feels unsafe	State limit and verification method	Credible useful answer
Diagram ignores operations	Focus is only happy path	Add telemetry, ownership, rollback, incident and DR	Operable design
Behavioral story is vague	No personal action or measurable result	Use STAR, metrics and learning	Clear contribution

Hands-on Lab 69: Run a Senior Delivery Engineering Mock Interview

Prerequisites

- A disposable source repository, CI platform, registry, and target environment.
- A second engineer or review role for approvals and independent validation.
- Synthetic data, short-lived identities, budgets, quotas, and cleanup automation.

Implementation

1. Select an enterprise delivery system-design prompt, troubleshooting incident, and four behavioral questions.
2. Spend five minutes clarifying outcomes, scale, security, reliability, migration, and team constraints.
3. Design source, CI, runners, artifact, promotion, GitOps/deployment, policy, observability, and recovery.
4. Have the interviewer change one constraint and revise the design.
5. Troubleshoot an incident using evidence and safe hypotheses.
6. Answer behavioral questions from an evidence-backed story bank.
7. Score clarity, depth, correctness, trade-offs, honesty, and time management; repeat the weakest section.

Validation Evidence

- Architecture, implementation, operations, recovery, and business outcome evidence are linked.
- A fresh reviewer can reproduce the normal path and at least one failure path.
- Residual risks, assumptions, owners, and next actions are explicit.

Cleanup

- Destroy temporary environments, runners, credentials, registries, artifacts, DNS, storage, and test records.
- Archive only sanitized code, diagrams, reports, dashboards, and lessons learned.

Interview Questions and Model Answers

1. What should happen before naming products?

Clarify outcomes, constraints, scale, trust, data, reliability, recovery, cost, and operating model.

2. How do you handle an unknown detail?

State what you know, identify decision risk, and explain how you would verify it.

3. What makes a good trade-off answer?

It compares credible alternatives against the scenario and explains consequences.

4. Why include operations in system design?

A design is incomplete if it cannot be deployed, observed, secured, recovered, and owned.

Chapter Summary

- Use structured answers and clarify first.
- Depth includes failure and recovery.
- Honesty improves technical credibility.
- Stories should contain evidence and learning.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Chapter 70: Final Enterprise Software Delivery Factory Capstone

Designing, delivering, operating, defending, recovering, and presenting a portfolio-grade delivery platform.

Why this chapter matters: The final capstone proves integrated engineering judgment across all Volume 10 capabilities and requires evidence that survives independent technical review.

Learning Objectives

- Explain capstone scenario and its business purpose.
- Design required architecture with explicit ownership and boundaries.
- Operate required evidence using measurable service levels.
- Troubleshoot delivery phases and defense through evidence and safe hypotheses.
- Produce portfolio evidence that can withstand technical review.

Core Concepts

Capstone scenario

A regulated digital-services company has Jenkins, GitHub Actions, GitLab CI/CD, several cloud accounts, Kubernetes clusters, legacy pipelines, public APIs, workers, and data services. It needs faster releases, controlled migration, tenant boundaries, supply-chain integrity, regional recovery, centralized evidence, and measurable platform economics.

Required architecture

Design source governance, reusable pipeline contracts, trust-tier runners, short-lived identity, dependency controls, artifacts, registry promotion, SBOM/provenance/signing, environment authorization, GitOps, progressive delivery, policy, telemetry, evidence, incident command, DR, and developer self-service.

Required evidence

Produce requirements, diagrams, ADRs, threat model, RACI, service catalog, repositories, reusable components, pipeline runs, artifact digests, attestations, policy tests, dashboards, SLOs, load results, migration evidence, incident timeline, rollback, clean-room recovery, cost model, user tests, and executive summary.

Delivery phases and defense

Phase the work through governance, platform foundations, application paths, security, operations, migration, failure testing, and presentation. Keep every phase deployable. Defend trade-offs to both technical and executive audiences and state residual risk honestly.

Architecture and Operating Decisions

- Use at least two CI platforms while standardizing one delivery contract.
- Include one Jenkins workload, one GitHub/GitLab reusable workflow, and one Kubernetes GitOps deployment.
- Build once and promote immutable artifacts with complete evidence.
- Run at least six failure scenarios and close defects through rebuild-and-retest cycles.
- Demonstrate measurable developer, reliability, security, and cost outcomes.

Reference Configuration or Command Pattern

```
delivery-factory/
  requirements/      # outcomes, SLOs, constraints, RACI
  architecture/     # diagrams, ADRs, threat and failure models
  components/       # Jenkins/GitHub/GitLab reusable delivery contracts
  runners/          # trust tiers, images, capacity and identity
  artifacts/        # promotion, SBOM, provenance and signing
  gitops/           # desired state, policy and progressive delivery
  operations/       # dashboards, alerts, incidents and DR
  migration/        # inventory, waves and retirement
  evidence/         # sanitized reproducible release records
  presentation/     # executive summary and technical defense
```

Production Readiness Checklist

- ☐ Ownership, trust boundaries, identities, inputs, outputs, and evidence are documented.
- ☐ Normal, negative, degraded, rollback, and recovery paths are tested.
- ☐ Changes are versioned, reviewed, reproducible, and independently verified.
- ☐ Secrets and privileged credentials are short-lived, least-privilege, and never embedded.
- ☐ Service levels, alerting, retention, exception handling, and cleanup are defined.

Troubleshooting Matrix

Symptom	Likely cause	Corrective action	Validation evidence
Architecture works only in happy path	Failure, capacity or ownership was deferred	Run game days and rebuild weak components	Closed defect evidence
Evidence cannot link release	Canonical identities are missing	Index source, workflow, artifact, desired state and runtime	Independent reconstruction
Platform is secure but unused	Developer journey and migration were ignored	Run user tests and remove top friction	Adoption and time-to-first-deploy
DR restores service but not trust	Compromised inputs were reused	Perform clean-room rebuild with new identities	New provenance and trusted release

Hands-on Lab 70: Deliver and Defend the Enterprise Software Delivery Factory

Prerequisites

- A disposable organization with at least two CI platforms, registry, Kubernetes sandbox, Git hosting, identity provider or safe equivalent, policy, telemetry, and evidence storage.
- Two sample services and one legacy pipeline.
- A review panel or independent engineer for user testing and technical defense.

Implementation

1. Define measurable outcomes, workload classes, risk tiers, environments, consumers, SLOs, RTO/RPO, compliance, cost, and acceptance criteria.
2. Build a vendor-neutral contract and implementations across Jenkins plus GitHub Actions or GitLab CI/CD.
3. Create isolated runner tiers, short-lived identity, artifact promotion, SBOM, provenance, signatures, policy, and evidence.
4. Deploy at least two services through Kubernetes GitOps, including progressive delivery and independent verification.
5. Migrate one legacy pipeline and retire its old execution authority.
6. Run queue saturation, dependency outage, bad configuration, wrong artifact, credential compromise, GitOps/controller loss, rollback, and clean-room recovery.
7. Present architecture, metrics, evidence, residual risk, roadmap, executive value, and a live technical defense.

Validation Evidence

- A fresh reviewer can bootstrap the platform and deliver one service.
- Release identity is traceable from source through runtime.
- The platform meets or explicitly misses measured SLO, recovery, security, and cost targets.
- Migration, incident, rollback, clean-room rebuild, and cleanup evidence is complete.
- Portfolio artifacts contain no credentials, private identifiers, customer data, or employer-sensitive information.

Cleanup

- Destroy organizations/projects, runners, clusters, registries, artifacts, identities, DNS, storage, logs, backups, and cost-bearing resources created for the capstone.
- Revoke every test credential and verify no privileged integration remains.
- Archive only sanitized source, diagrams, reports, dashboards, runbooks, evidence, and lessons learned.

Interview Questions and Model Answers

1. What is the capstone's most important outcome?

Reproducible evidence that architecture, delivery, security, operation, migration, recovery, cost, and user value work together.

2. How should residual risk be presented?

Likelihood, impact, scope, evidence, compensating controls, owner, approval, expiry or review trigger, and planned reduction.

3. What makes a portfolio credible?

Deployable code, measured tests, honest trade-offs, sanitized evidence, clear ownership, and successful recovery.

4. What demonstrates senior delivery engineering?

Consistent decisions across business outcomes, trust, flow, reliability, security, operations, economics, people, and change over time.

Chapter Summary

- The capstone integrates every Volume 10 operating discipline.
- Every claim must link to reproducible evidence.
- Failure and recovery tests reveal architecture quality.
- Technical depth and business value must both be communicated.

Completion evidence: Record the repository revision, execution or test output, decision record, recovery result, and sanitized screenshots or logs before marking this chapter complete.

Part F Command and Investigation Reference

Purpose	Reference pattern
Git revision	git rev-parse HEAD && git log -1 --show-signature
Jenkins system info	Use the controller system information and support bundle with secret-safe handling
Jenkins queue	Inspect queue, executors, node labels, build causes and pipeline stage timing
GitHub run	gh run view <run-id> --log-failed
GitHub workflow	gh workflow view <workflow> && gh run list --workflow <workflow>
GitLab pipeline	Use project pipeline/job APIs or glab with scoped read-only token
Kubernetes rollout	kubectl rollout status deployment/<name> -n <ns> --timeout=5m
Argo CD status	argocd app get <app> --refresh
Flux status	flux get all -A
Artifact identity	Resolve registry reference to sha256 digest before promotion
Evidence hash	sha256sum <artifact-or-report>
Incident timeline	Record UTC timestamp, actor, observation, action, result and next decision

Enterprise Delivery Graduation Checklist

- ☐ Business outcome, consumers, service owner, platform owner, risk owner, on-call, and cost owner are named.
- ☐ Source, workflow, runner, identity, artifact, environment, policy, and evidence trust boundaries are diagrammed.
- ☐ Pipelines use deterministic inputs, isolated execution, immutable artifacts, promotion, verification, and recovery.
- ☐ Jenkins, GitHub Actions, GitLab CI/CD, and GitOps responsibilities are standardized or intentionally bounded.
- ☐ Delivery SLOs cover availability, queue, feedback, release, reconciliation, verification, and recovery.
- ☐ Controller, runner, registry, identity, policy, and evidence dependencies have tested continuity plans.
- ☐ Security controls include SBOM, provenance, signing, policy, contextual risk, exceptions, and incident response.
- ☐ Platform capacity, quotas, cost, upgrade, deprecation, support, and adoption are measured.
- ☐ Portfolio artifacts are reproducible, sanitized, honest about residual risk, and linked to evidence.
- ☐ A fresh reviewer can deploy, break, recover, and explain the final capstone.

Ten Additional Portfolio Projects

Project	Primary evidence	Difficulty
1. Unified pipeline component library	three-platform implementation and compatibility tests	Advanced

Project	Primary evidence	Difficulty
2. Pipeline migration factory	inventory, wave plan, automated conversion and rollback	Advanced
3. Release train control plane	calendar, dependencies, evidence and emergency release	Advanced
4. Multi-cluster GitOps platform	promotion, tenancy, drift, failover and SLOs	Advanced
5. Runner capacity service	queue model, autoscaling, quotas and cost dashboard	Advanced
6. Artifact trust service	registry promotion, SBOM, provenance and signatures	Advanced
7. Delivery incident command lab	timeline, hypotheses, mitigation and prevention	Medium
8. Clean-room delivery recovery	controller/runner rebuild and trusted release	Advanced
9. Internal developer platform	service catalog, golden paths and adoption research	Advanced
10. Delivery analytics product	DORA-style metrics, SLOs, quality and cost insights	Medium

90-Day Enterprise Delivery Practice Plan

Weeks	Focus	Required output
1-2	Architecture, portfolio and value	operating model, opportunity inventory and ADRs
3-4	Multi-platform standardization and Kubernetes project	reusable components and GitOps platform
5-6	Promotion, release trains and brownfield migration	release governance and migration wave evidence
7-8	DR and incident response	controller, runner, identity and supply-chain game days
9-10	Performance and platform product	capacity model, SLOs, adoption and roadmap
11	Certifications and interviews	gap matrix, practice labs and mock interviews
12-13	Final delivery factory	integrated capstone, portfolio and technical defense

Glossary

Term	Definition
Blast radius	Maximum scope of impact from a fault, change, compromise, or operational action.

Term	Definition
Change failure rate	Share of production changes requiring remediation, rollback, fix-forward, or incident response.
Control plane	Systems that authorize, schedule, configure, or reconcile delivery activity.
Deployment frequency	Rate at which production changes are successfully delivered.
Error budget	Allowed unreliability derived from an SLO.
Golden path	Supported end-to-end platform route for a common workload.
Lead time for changes	Elapsed time from change creation or commit to successful production operation.
MTTR	Time to restore service or trusted delivery capability after failure.
Platform product	Internal service with users, interfaces, SLOs, support, roadmap, and lifecycle.
Release train	Coordinated schedule and governance for related changes.
RPO	Maximum acceptable data or state loss measured in time.
RTO	Target duration to restore service or platform capability.

Official Documentation Index

Jenkins Documentation: <https://www.jenkins.io/doc/>

Jenkins Pipeline: <https://www.jenkins.io/doc/book/pipeline/>

GitHub Actions: <https://docs.github.com/actions>

GitHub Actions Security: <https://docs.github.com/actions/security-guides>

GitLab CI/CD: <https://docs.gitlab.com/ci/>

GitLab Runner: <https://docs.gitlab.com/runner/>

Kubernetes Documentation: <https://kubernetes.io/docs/>

Argo CD: <https://argo-cd.readthedocs.io/>

Flux: <https://fluxcd.io/flux/>

Argo Rollouts: <https://argo-rollouts.readthedocs.io/>

SLSA: <https://slsa.dev/>

Linux Foundation Training and Certification: <https://training.linuxfoundation.org/>

GitHub Certifications: <https://resources.github.com/learn/certifications/>

GitLab Certifications: <https://university.gitlab.com/pages/certifications>

Verification note: Products, versions, certification blueprints, pricing, quotas, and feature status change. Check the current official documentation before production use or exam registration.

Volume 10 Completion

With Part F complete, Volume 10 spans delivery foundations, Jenkins, GitHub Actions, GitLab CI/CD, Kubernetes GitOps, progressive delivery, DevSecOps, supply-chain integrity, enterprise operations, certifications, interviews, and a final portfolio-grade software-delivery factory.

Final action: Complete the labs, retain sanitized evidence, score at least 80% on the knowledge checks, and rehearse the recovery procedures before marking this part complete.